PROYECTO DE PRÁCTICA · PATRONES DE INTEGRACIÓN

Portal de Información

OmniView — un portal agregador sobre cuatro sistemas heterogéneos, en Java y Spring Boot

---

Autor: Lucas Blanco

Fecha: 21 de septiembre de 2026

Duración estimada: 6 semanas · nivel avanzado

Basado en: el patrón *Information Portal* de Hohpe & Woolf

Contexto

El patrón *Information Portal* resuelve un problema de integración a nivel de presentación: el usuario necesita datos que viven en sistemas distintos y, sin portal, tiene que abrir una ventana por sistema.

La integración ocurre en la capa de UI, no en la de datos. Eso lo hace ideal para practicar: no hay que migrar ni sincronizar bases, pero sí hay que resolver los problemas reales de todo agregador — protocolos heterogéneos, latencias distintas, fallas parciales, correlación de identificadores entre sistemas.

El texto distingue tres niveles de sofisticación, y el proyecto recorre los tres:

1. **Portal simple** — la pantalla se divide en zonas, cada una muestra datos de un sistema, sin comunicación entre zonas.
2. **Portal con interacción limitada** — seleccionar un ítem en la zona A refresca la zona B con el detalle correspondiente.
3. **Portal integrado** — la interacción es tan rica que la frontera entre portal y aplicación propia se difumina.

> El salto de 1 a 2 es donde aparece el trabajo interesante: para que una zona reaccione a otra hay que tener un modelo de correlación común (¿el `customerId` del mainframe es el mismo que el del sistema web?) y una estrategia de refresco que no rompa cuando un sistema tarda o se cae.

Enunciado

Construir **OmniView**, el portal de información que usa el área de atención al cliente de *Nordia*, un retailer que vende por tres canales y arrastra treinta años de sistemas heredados.

Hoy, para responder «¿dónde está mi pedido?», un representante abre cuatro ventanas: el emulador 3270 del mainframe donde viven los pedidos de tienda y catálogo, el back-office web donde viven los pedidos online, el sistema de logística del operador tercerizado, y el CRM donde están los reclamos abiertos. El promedio de manejo por llamada es de 6 minutos y medio, y más de la mitad se va en navegar entre sistemas.

El proyecto pide una única pantalla que, dado un cliente, muestre todo eso junto, con interacción entre zonas, y que siga siendo usable cuando uno de los sistemas fuente no responde.

Restricciones de diseño

Son parte del ejercicio y no se negocian:

- **No se puede modificar ningún sistema fuente.** Se consumen como están, con los protocolos que exponen.

- **No se replica la base de datos de nadie.** La integración es de lectura y en tiempo de consulta; lo único que se persiste es caché con TTL y datos propios del portal (preferencias, auditoría).
- **El portal es de solo lectura** en las fases 1 a 4. La escritura aparece recién como extensión opcional, y con un patrón distinto.
- **Cada sistema fuente identifica al cliente a su manera.** Resolver la correlación es parte del trabajo, no un detalle.

Arquitectura

Un BFF (*backend for frontend*) concentra la agregación: el navegador hace una sola llamada por zona, y el BFF traduce, correlaciona y compone. Los adaptadores aíslan cada protocolo detrás de una interfaz de dominio uniforme.

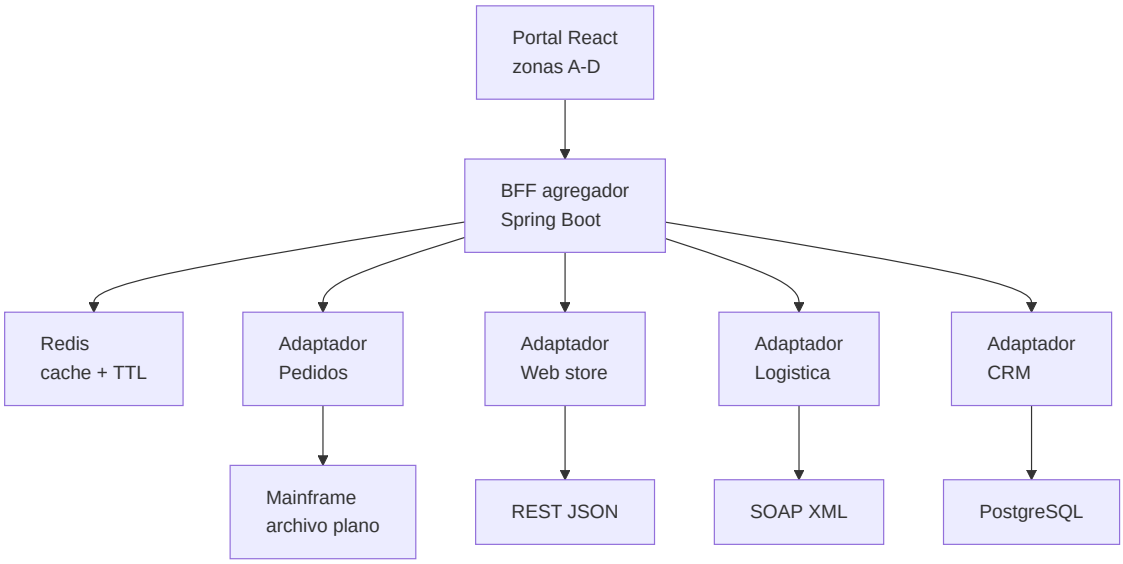


Figura 1 — Componentes de OmniView: un adaptador por protocolo, un BFF que agrega.

Cada adaptador devuelve un tipo del dominio del portal (`Pedido`, `Envío`, `Reclamo`), nunca el DTO del sistema fuente. Esa frontera es la que permite cambiar un sistema sin tocar la agregación ni la UI.

Decisiones que hay que justificar en el informe

Decisión	Opciones	Qué evaluar
Agregación	Un endpoint compuesto vs. un endpoint por zona	Latencia percibida vs. acoplamiento del front
Concurrencia	`CompletableFuture` vs. WebFlux reactivo	Uso de hilos con fuentes lentas
Correlación de IDs	Servicio de identidad propio vs. lookup en cascada	Costo de la consulta extra vs. consistencia
Caché	Por fuente vs. de la respuesta compuesta	Invalidación y frescura por tipo de dato
Falla parcial	Zona vacía vs. dato stale marcado	Qué espera el representante en el teléfono

Sistemas fuente a simular

Los cuatro se implementan como servicios independientes, cada uno en su contenedor. La heterogeneidad es deliberada: si los cuatro hablaran REST JSON el ejercicio perdería su razón de ser.

Sistema	Protocolo	Identifica al cliente por	Latencia simulada	Comportamiento adverso
Order Management (mainframe)	Archivo plano de ancho fijo vía FTP/SFTP o endpoint que lo devuelve como texto	`CUST-NBR` numérico de 8 dígitos	800–2000 ms	Se cae los martes 3:00–3:20 (ventana batch)
Web Store	REST JSON, paginado	`customerId` UUID	50–150 ms	Devuelve 429 si superás 10 req/s
Logística (tercero)	SOAP 1.1 con WSDL	Número de tracking + email	300–1500 ms	5% de las llamadas devuelve timeout
CRM	PostgreSQL, acceso directo por JDBC	Email	20–80 ms	Sin fallas, pero el esquema es feo

Dos detalles que parecen crueles y son el punto del ejercicio:

- El **archivo plano** no trae separadores. Hay que escribir un parser posicional a partir de un *copybook* documentado: posiciones 1–8 el número de cliente, 9–20 el número de pedido, 21–28 la fecha en `YYYYMMDD`, 29–38 el importe con dos decimales implícitos y signo *overpunch*. Spring Batch (`FlatFileItemReader` con `FixedLengthTokenizer`) hace el trabajo pesado.
- La **correlación de identidad** no está resuelta en ningún lado. Hay que construir un `CustomerIdentityService` que, dado un email, resuelva el UUID web y el `CUST-NBR` del mainframe — con su propio caché, porque lo consultan las cuatro zonas.

Cada mock debe exponer su latencia y su tasa de error como configuración, para poder forzar escenarios en los tests de resiliencia. WireMock y `toxiproxy` sirven para esto sin escribir los mocks a mano.

Requisitos funcionales

Las cuatro zonas

Zona	Contenido	Fuente	Frescura aceptable
A — Cliente	Datos de contacto, segmento, antigüedad, IDs resueltos en cada sistema	CRM + servicio de identidad	5 min
B — Pedidos	Lista unificada de pedidos de los tres canales, ordenada por fecha desc	Mainframe + Web Store	2 min
C — Envío	Estado y eventos de tracking del pedido seleccionado en B	Logística	30 s
D — Reclamos	Tickets abiertos del cliente, o del pedido seleccionado si hay uno	CRM	1 min

Interacción entre zonas

Es el requisito central, el que convierte un tablero estático en un portal de verdad.

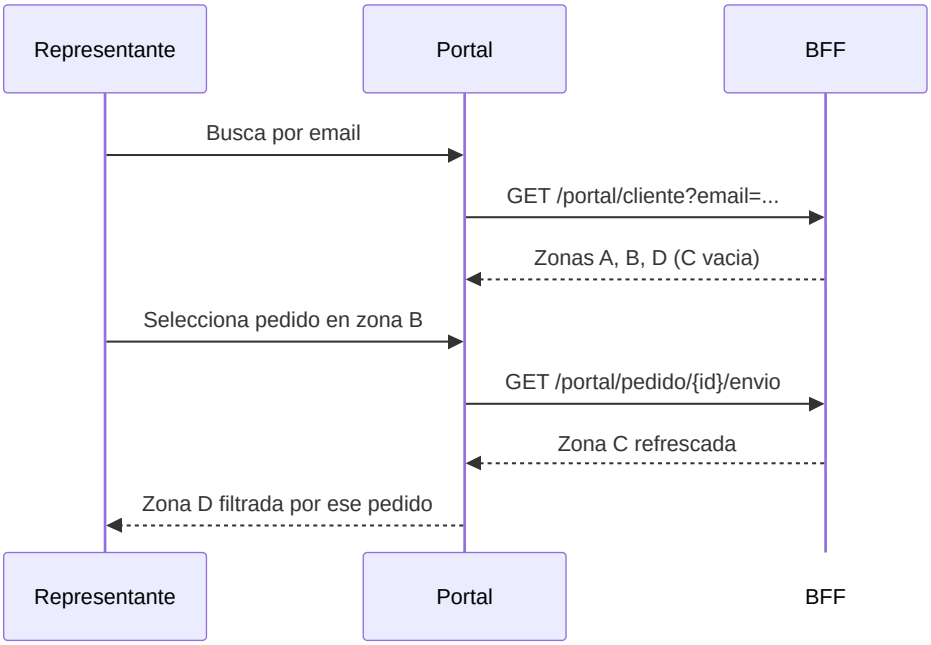


Figura 2 — Carga inicial en paralelo, y el refresco de C y D disparado por la selección en B.

Reglas concretas:

1. Al cargar, las zonas A, B y D se resuelven **en paralelo**; C queda en estado vacío con un mensaje que invita a elegir un pedido.

2. Seleccionar un pedido en B refresca C y **filtra** D, sin recargar A ni B.
3. Deseleccionar devuelve D a su estado sin filtro. El estado de selección vive en la URL, para que el representante pueda compartir el enlace exacto con un supervisor.
4. Cada zona muestra su propia marca de tiempo de última actualización y su propio botón de refresco.

Búsqueda unificada

Un solo campo acepta email, número de pedido de cualquier canal, o número de tracking. El portal infiere el tipo por el formato, resuelve la identidad y arma la vista. Si el término matchea en más de un sistema, se muestra un desambiguador antes de cargar las zonas.

Requisitos no funcionales

Acá es donde el proyecto se vuelve avanzado. Un agregador es tan lento como su fuente más lenta y tan frágil como su fuente más caída, salvo que se diseñe para lo contrario.

Rendimiento

- Las llamadas a fuentes distintas se hacen en paralelo, nunca en secuencia. Con las latencias de la tabla anterior, secuencial da ~3,7 s en el peor caso y paralelo ~2 s.
- Presupuesto de latencia: **p95 por debajo de 2,5 s** para la carga inicial completa, **p95 por debajo de 800 ms** para el refresco de una zona. El p95 es el percentil 95: solo 1 de cada 20 cargas puede pasarse de ese umbral.
- El portal pinta cada zona apenas llega su dato. Nada espera a que terminen las cuatro.

Resiliencia

- Timeout por fuente, más corto que el del navegador: 3 s para el mainframe, 1 s para el resto.
- Circuit breaker por fuente (Resilience4j), con umbral de 50% de fallas sobre ventana de 20 llamadas y 30 s en estado abierto.
- Reintento con backoff exponencial solo en errores transitorios e idempotentes. Un 429 del Web Store se reintenta respetando `Retry-After`; un 500 del CRM no.
- **Degradación parcial obligatoria**: si una fuente cae, su zona muestra el último valor cacheado con la marca visible «datos de hace N minutos», o un estado de error acotado a esa zona. El resto del portal sigue funcionando. Que una zona rota tire abajo la pantalla entera es causa de rechazo del entregable.
- Bulkhead: el pool de hilos del mainframe no puede agotar los del resto.

Caché

TTL por tipo de dato, según la columna de frescura de la sección anterior. Clave compuesta por fuente + identificador + versión del adaptador, para que un deploy invalide lo viejo. Redis como store, con `@Cacheable` y un `CacheManager` por TTL.

Seguridad

- Autenticación OIDC contra Keycloak; el portal es un *confidential client*.
- Propagación de identidad hacia las fuentes que la soportan; credenciales técnicas para las que no (el mainframe), nunca en el código.
- Autorización por rol: `AGENTE` ve todo menos el historial de pagos, `SUPERVISOR` ve todo.
- Auditoría: cada consulta de un cliente queda registrada con usuario, cliente consultado y timestamp. Es un portal de datos personales.

Observabilidad

Trazas distribuidas con Micrometer Tracing y OpenTelemetry, de modo que un span padre por request tenga un span hijo por fuente. Sin eso no se puede responder «¿qué zona está haciendo lenta la pantalla?», que es la pregunta que más se va a hacer. Dashboard con p95 por fuente, tasa de acierto de caché y estado de cada circuit breaker.

Stack y dependencias

Capa	Elección	Por qué
Runtime	Java 21, Spring Boot 3.3+	Virtual threads resuelven el fan-out bloqueante sin reescribir a reactivo
Cliente HTTP	`RestClient` o `WebClient`	`WebClient` si vas por WebFlux; `RestClient` + virtual threads si vas imperativo
Resiliencia	`resilience4j-spring-boot3`	Circuit breaker, retry, bulkhead y timelimiter por anotación
SOAP	`spring-ws-core` + `maven-jaxb2-plugin`	Genera los stubs desde el WSDL del mock de logística
Archivo plano	`spring-batch-infrastructure`	`FixedLengthTokenizer` para el copybook
Caché	`spring-boot-starter-data-redis`	TTL por cache, serialización JSON
Seguridad	`spring-boot-starter-oauth2-client` + Keycloak	OIDC sin escribir el flujo a mano
Front	React 18 + TanStack Query	Una query por zona, con `staleTime` propio y refetch independiente
Tests	JUnit 5, Testcontainers, WireMock	Fuentes reales en contenedor para los tests de integración
Orquestación	Docker Compose	Nueve contenedores: portal, 4 mocks, Redis, Postgres, Keycloak, Grafana

> **Una nota sobre virtual threads:** con `spring.threads.virtual.enabled=true` podés hacer fan-out con `CompletableFuture` y clientes bloqueantes sin pagar el costo de hilos de plataforma. Es el camino más corto a un agregador correcto en Spring Boot hoy, y evita reescribir todo el stack a WebFlux para ganar concurrencia.

Plan por fases

Seis fases de una semana cada una. Cada una termina con algo que corre, no con un documento.

#	Fase	Entregable que corre
1	Mocks y andamiaje	Los 4 sistemas fuente levantan con Docker Compose y responden con datos semilla coherentes entre sí
2	Adaptadores	Un test de integración por fuente, con Testcontainers, que trae datos y los mapea al dominio del portal
3	Agregación e identidad	`GET /portal/cliente?email=` devuelve las 4 zonas, en paralelo, con IDs correlacionados
4	Portal y zonas	UI con las 4 zonas, interacción B→C→D, estado en la URL, refresco por zona
5	Resiliencia y caché	Circuit breakers, TTLs, degradación parcial demostrada apagando un mock en vivo
6	Seguridad y observabilidad	Login OIDC, roles, auditoría, trazas end-to-end y dashboard de latencia por fuente

La tentación natural es empezar por la UI. No lo hagas: sin datos semilla coherentes entre los cuatro sistemas — el mismo cliente con cuatro identificadores distintos y pedidos que se cruzan — la fase 3 no tiene cómo probarse. La fase 1 es la que decide si el resto del proyecto enseña algo.

Entregables y evaluación

Qué se entrega

- Monorepo con `docker-compose up` como único comando de arranque, y datos semilla cargados automáticamente.
- README con el diagrama de arquitectura y las decisiones de la tabla de la sección *Arquitectura*, cada una con su justificación en dos o tres líneas.
- Suite de tests: unitarios de los mapeos, de integración por adaptador con Testcontainers, y de resiliencia con fuentes degradadas.
- Un video o GIF de 2 minutos apagando un mock en vivo y mostrando que el portal sigue en pie con la zona degradada.

Rúbrica

Criterio	Peso	Qué se mira
Aislamiento de fuentes	20%	Ningún DTO de un sistema fuente cruza al dominio del portal ni a la UI
Agregación y latencia	20%	Paralelismo real medible en las trazas; p95 dentro del presupuesto
Degradación parcial	20%	Apagar cualquier fuente deja el portal usable, con el estado visible en la zona afectada
Interacción entre zonas	15%	B→C y B→D funcionan sin recargar el resto; el estado vive en la URL
Correlación de identidad	10%	Un solo servicio resuelve los IDs, cacheado, testeado con casos sin match
Tests	10%	Los de resiliencia prueban fallas reales, no mocks del circuit breaker
Seguridad y auditoría	5%	Roles aplicados y consultas auditadas

Criterios de rechazo

Tres cosas invalidan el entregable por más prolijo que esté el resto: que una fuente caída rompa la pantalla entera, que la agregación sea secuencial, y que el portal escriba en la base de un sistema fuente.

Extensiones opcionales

Cada una empuja el portal hacia el tercer nivel del texto original, donde la frontera entre portal y aplicación se difumina.

1. **Portal compuesto con micro-frontends.** Cada zona se despliega por separado con Module Federation. Aparece el problema real de los portales corporativos: versionado independiente y contrato compartido entre zonas.
2. **Push en lugar de polling.** Los eventos de tracking llegan por WebSocket o SSE y la zona C se actualiza sola. El portal pasa de *pull* a *push* y hay que decidir qué hacer con el caché.
3. **Escritura con Compensating Transaction.** Permitir cancelar un pedido desde el portal, cuando la cancelación toca dos sistemas que no comparten transacción. Es un patrón distinto — saga con compensación — y merece su propio diseño.
4. **GraphQL federado** como capa de agregación, con un subgrafo por adaptador. Compará medido contra el BFF REST: el over-fetching baja, la complejidad operativa sube.
5. **Read model con CQRS.** Materializar una vista de cliente alimentada por eventos, y contrastarla con la agregación en tiempo de consulta. Es exactamente el debate *Information Portal* vs. *Shared Database* del libro, ahora con números propios de latencia y frescura para defender la elección.

Fuente. El texto de partida es la descripción del patrón *Information Portal* de Gregor Hohpe y Bobby Woolf, *Enterprise Integration Patterns* (Addison-Wesley, 2003), en el capítulo introductorio sobre estilos y criterios de integración.